

Risposte alle domande per l'Esame Orale - Algoritmi e Strutture Dati

Paolo Bianchessi

13 febbraio 2026

Indice

1	Notazione usata (coerente con il corso)	1
2	Sigle e acronimi (definiti una volta)	1
3	Pseudocodici principali (stile prof)	2
3.1	Dijkstra	2
3.2	Kruskal	2
3.3	QuickSort in-place	3
3.4	HeapSort	3
4	Grafi	3
5	Alberi	6
6	Algoritmi di ordinamento	7
7	Strutture dati	8
8	Analisi degli algoritmi	9
9	Mini-schede di complessità utili per ripasso	10

1 Notazione usata (coerente con il corso)

- Grafi pesati: $G = (V, E, \omega)$, con $n = |V|$, $m = |E|$.
- Distanze/costi: vettore D o d indicizzato da V , predecessori $pred$, valore infinito $+\infty$.
- Insiemi disgiunti: operazioni MAKESET, FIND, UNION.
- Heap: vettore posizionale, figlio sinistro di i in $2i + 1$, destro in $2i + 2$.

2 Sigle e acronimi (definiti una volta)

- **AVL**: albero bilanciato in altezza (Adelson-Velsky e Landis).
- **BFS/DFS**: visita in ampiezza / visita in profondità.
- **MST**: albero ricoprente minimo (*Minimum Spanning Tree*).
- **MSF**: foresta ricoprente minima (*Minimum Spanning Forest*).

- **FIFO**: *First In, First Out*.
- **BF** (in alberi AVL): fattore di bilanciamento.

3 Pseudocodici principali (stile prof)

3.1 Dijkstra

Algorithm 1 Dijkstra (schema classico)

```

1: procedure DIJKSTRA(grafo pesato  $G = (V, E, \omega), s) \rightarrow$  vettore distanze
2:   Sia  $D$  un vettore con insieme di indici  $V$ 
3:   Sia  $pred$  un vettore con insieme di indici  $V$ 
4:    $D[s] \leftarrow 0$ 
5:   for all  $v \in V \setminus \{s\}$  do
6:      $D[v] \leftarrow +\infty$ 
7:    $C \leftarrow V$ 
8:   while  $C \neq \emptyset$  do
9:      $u \leftarrow$  elemento di  $C$  con  $D[u]$  minima
10:     $C \leftarrow C \setminus \{u\}$ 
11:    for all  $(u, v) \in E$  do
12:      if  $D[u] + \omega(u, v) < D[v]$  then
13:         $D[v] \leftarrow D[u] + \omega(u, v)$ 
14:         $pred[v] \leftarrow u$ 
15:   return  $D$ 

```

3.2 Kruskal

Algorithm 2 Kruskal (con Union-Find)

```

1: procedure KRUSKAL(grafo connesso non orientato  $G = (V, E, \omega)) \rightarrow$  albero
2:   ordina gli archi  $E$  per peso crescente
3:    $T \leftarrow (V, \emptyset)$ 
4:   Sia  $P$  una partizione inizialmente vuota
5:   for all  $v \in V$  do
6:     P.MAKESET( $v$ )
7:   for all  $(u, v) \in E$  in ordine do
8:      $T_u \leftarrow$  P.FIND( $u$ )
9:      $T_v \leftarrow$  P.FIND( $v$ )
10:    if  $T_u \neq T_v$  then
11:       $T \leftarrow T \cup \{(u, v)\}$ 
12:      P.UNION( $T_u, T_v$ )
13:   return  $T$ 

```

3.3 QuickSort in-place

Algorithm 3 QuickSort (in-place, schema)

```
1: procedure QUICKSORT( $A, \ell, r$ )
2:   if  $\ell \geq r$  then
3:     return
4:    $p \leftarrow \text{PARTITION}(A, \ell, r)$ 
5:   QUICKSORT( $A, \ell, p - 1$ )
6:   QUICKSORT( $A, p + 1, r$ )
```

3.4 HeapSort

Algorithm 4 HeapSort (schema)

```
1: procedure HEAPSORT( $A$ )
2:   BUILDMAXHEAP( $A$ )
3:   for  $end \leftarrow n - 1$  downto 1 do
4:     scambia  $A[0]$  e  $A[end]$ 
5:     FIXHEAP( $A, 0, end - 1$ )
```

4 Grafi

1. Incidenza

Un arco (o spigolo) (x, y) si dice **incidente** sui suoi vertici (nodi) di estremità x e y

2. Adiacenza

- **Grafi non orientati**: Se esiste un arco (x, y) , i vertici x e y sono **adiacenti**. La relazione è reciproca.
- **Grafi orientati**: Se esiste un arco (x, y) che **esce da x ed entra in y** , si dice che y è **adiacente** a x . La relazione non è reciproca: x è adiacente a y solo se esiste anche un arco (y, x) .

3. Vicini di $v \in V$: vertici adiacenti a v

4. Grado $\delta(v)$ di un vertice: numero di archi adiacenti su v

- $\delta_{in}(v)$ grado in **ingresso**
- $\delta_{out}(v)$ grado in **uscita**
- $\delta(v) = \delta_{in}(v) + \delta_{out}(v)$
- $\sum_v \delta_{in}(v) = m = \sum_v \delta_{out}(v)$

5. Cammino da x a y ($x, y \in V$)

- sequenza di vertici $v_0, v_1, \dots, v_k \in V$ tale che $v_0 = x$, $v_k = y$ e $v_{i-1}, v_i \in E$ per $i = 1 \dots k$
- lunghezza del cammino = $\#archi(k)$
- un cammino **semplice** non contiene vertici ripetuti
- affermiamo che y è raggiungibile da x se $\exists$ cammino da x e y

6. Ciclo: cammino da un vertice v a v stesso.

- **ciclo semplice** solo il vertice iniziale è ripetuto (alla fine)

7. **Catena** tra x e y ($x, y \in V$):

- sequenza di vertici $v_0, v_1, \dots, v_k \in V$ tale che $v_0 = x, v_k = y$ e $((v_{i-1}, v_i) \in E \text{ o } (v_i, v_{i-1}) \in E$ per $i = 1 \dots k$
- **circuito** se $x = y$

8. **Grafo connesso**: tra ogni coppia di vertici esiste una catena

9. **Cos'è un grafo fortemente connesso?**

tra ogni coppia di vertici esiste un cammino

- $G'' = (V', E')$ è un sottografo di $G = (V, E)$ quando $V' \subseteq V$ e $E' \subseteq E \cap (V' \times V')$
- G'' è il sottografo indotto da V' quando $E' = E \cap (V' \times V')$

Una **componente fortemente connessa** è data dal sottografo indotto fortemente connesso massimale.

10. **Le rappresentazioni possibili di un grafo.**

- **Lista di archi** rappresento solo gli archi in un array. $\Theta(m)$ spazio totale
- **Lista di adiacenza** per ogni vertice lista di vertici adiacenti - lunghezza di ciascuna lista = grado vertice corrispondente
 - **per grafi non orientati** la lunghezza di ciascuna lista è pari al grado del vertice corrispondente $\sum$ lunghezza delle liste = $2m$
 - **per grafi orientati** lunghezza lista associata al vertice $v = \delta_{out}(v)$; $\sum$ lunghezza delle liste = $\sum_v \delta_{out}(v) = m$
- **Lista di incidenza** ideale quando gli archi portano molte informazioni (peso, colore, ...). Array di vertici dove ogni cella punta a una lista di indici riferiti a un array separato contenente gli archi. Spazio $\Theta(n+m)$, utile per associare dati agli archi senza duplicarli.
- **Matrice di adiacenza**, valori *booleani*, gli indici rappresentano i vertici del grafo. $M[u, v] = 1 \iff (u, v) \in E$. Per un grafo non orientato la matrice sarà simmetrica. Spazio $\Theta(n^2)$
- **Matrice di incidenza** Matrice $n \times m$ (righe=vertici, colonne=archi). La cella $M[i, j] \neq 0$ se il vertice i è un estremo dell'arco j . Ogni colonna ha solo due valori non nulli. Spazio $\Theta(n \cdot m)$.

11. **Come rappresentare il grafo (es. lista di adiacenza, lista di archi, matrice di incidenza).**

Lista adiacenza: per ogni vertice memorizzi vicini (e peso). Lista archi: coppie/triple (u, v, ω) . Matrice incidenza: righe vertici, colonne archi con valori di incidenza.

12. **I problemi dei cammini minimi nei grafi.**

Da una sorgente a tutti i vertici, tra una coppia di vertici, oppure tra tutte le coppie di vertici, con o senza pesi negativi.

13. **Problema di cammino minimo (definizione e varianti, come distanza e cammino minimo fra tutti i vertici).**

Dato $G = (V, E, \omega)$, minimizzare la somma dei pesi lungo un cammino. Varianti: da una sorgente, su tutte le coppie, e con vincoli (es. numero massimo di archi).

14. **Algoritmo di Dijkstra (ad alto livello, strategia greedy, spiegazione dettagliata e pseudocodice).**

Greedy: a ogni passo "fissa" il vertice non ancora scelto con distanza provvisoria minima. Corretto se $\omega(e) \geq 0$ per ogni arco.

15. **Che operazioni si fanno sulla coda di priorità in Dijkstra?**
EXTRACT-MIN, DECREASE-KEY, eventuale INSERT iniziale dei vertici. Con heap binario: tipicamente $\mathcal{O}((n + m) \log n)$.
16. **Kruskal applicato a un grafo non connesso: cosa si ottiene?**
Si ottiene una *foresta ricoprente minima*, cioè un albero minimo per ogni componente connessa.
17. **Differenze fra Kruskal e Prim (tecniche greedy).**
Kruskal sceglie globalmente il prossimo arco minimo che non crea cicli. Prim cresce un unico albero da un vertice aggiungendo il miglior arco uscente dal taglio corrente.
18. **Algoritmi per trovare l'albero ricoprente minimo: Kruskal e Prim.**
Entrambi sono greedy e corretti per grafi non orientati pesati; Kruskal usa insiemi disgiunti (MAKESET/FIND/UNION), Prim usa una coda con priorità sui vertici di frontiera.
19. **Spiegazione dell'algoritmo di Kruskal.**
Ordina E per peso; inizializza ogni vertice in un insieme separato; aggiunge (u, v) solo se $\text{FIND}(u) \neq \text{FIND}(v)$; unisce i due insiemi.
20. **Come cambia la complessità di Kruskal usando BubbleSort per ordinare gli archi.**
L'ordinamento passa a $\Theta(m^2)$ invece di $\Theta(m \log m)$, quindi l'intero algoritmo diventa dominato da $\Theta(m^2)$.
21. **Algoritmi greedy che trovano sempre la soluzione ottima: Kruskal, Prim, Dijkstra.**
Kruskal e Prim: sempre ottimi per il problema dell'albero ricoprente minimo (MST). Dijkstra: ottimo solo con pesi non negativi.
22. **Definizione di albero ricoprente minimo.**
Albero ricoprente di un grafo connesso non orientato con somma pesi minima tra tutti i ricoprenti.
23. **Rappresentazione di un grafo per Kruskal.**
La più naturale è lista di archi (u, v, ω) , perché il passo chiave è ordinare gli archi.
24. **Differenza tra lista di adiacenza e lista di archi.**
Lista adiacenza facilita scansione dei vicini di un nodo; lista archi facilita operazioni globali sugli archi (es. sort in Kruskal).
25. **Distanza da tutti i vertici a un singolo vertice.**
Su grafo orientato si risolve con Dijkstra/Bellman-Ford sul grafo trasposto, usando quel vertice come sorgente.
26. **Algoritmi di cammini minimi: Dijkstra, Bellman-Ford, Floyd-Warshall.**
Dijkstra: cammini minimi da una sorgente con pesi non negativi. Bellman-Ford: cammini minimi da una sorgente con pesi anche negativi e rilevazione di cicli negativi. Floyd-Warshall: cammini minimi tra tutte le coppie in $\Theta(n^3)$.
27. **Differenza tra Dijkstra e Bellman-Ford.**
Dijkstra è greedy più veloce, ma richiede pesi non negativi. Bellman-Ford rilassa tutti gli archi $n - 1$ volte: più lento, ma gestisce negativi.
28. **Problemi con cicli negativi negli algoritmi di cammini minimi.**
Se un ciclo negativo è raggiungibile dalla sorgente, la distanza minima non è ben definita $(-\infty)$. Bellman-Ford lo segnala.

29. Matrice di adiacenza e trasformazione tra rappresentazioni.

Da matrice a lista: scansione completa $\Theta(n^2)$. Da lista a matrice: inizializza matrice e inserisci archi in $\Theta(n^2)$ (o $\Theta(n + m)$ se matrice già allocata).

30. Cosa garantisce che un grafo sia connesso (tra ogni coppia di vertici esiste un cammino).

Per grafi non orientati: esistenza di un unico componente connesso, equivalentemente una visita in ampiezza o in profondità da un nodo visita tutti i vertici.

31. Relazione tra vertici e archi in un albero.

Per un albero con n vertici vale sempre $m = n - 1$.

5 Alberi

1. Cos'è un albero ricoprente?

Sottografo con tutti i vertici del grafo originale, connesso e aciclico.

2. Cos'è un albero ricoprente minimo?

Albero ricoprente con peso totale minimo.

3. Gli algoritmi per trovare l'albero ricoprente minimo (Prim e Kruskal).

Prim cresce un albero ricoprente minimo da un nodo; Kruskal unisce componenti via archi ordinati e struttura a insiemi disgiunti.

4. Alberi come struttura dati (binari di ricerca, 2-3, B-alberi).

Albero binario di ricerca: ordine sinistra/destra. AVL: albero binario di ricerca bilanciato in altezza. Alberi 2-3 e B-alberi: strutture multi-way bilanciate, utili anche su memoria secondaria.

5. Cos'è un albero binario di ricerca?

Per ogni nodo n : chiavi nel sottoalbero sinistro $< key(n)$, nel destro $\geq key(n)$; visita simmetrica produce sequenza ordinata.

6. Le condizioni di bilanciamento degli alberi binari.

Perfettamente bilanciato: differenza tra numero di nodi dei sottoalberi al più 1. AVL: differenza tra altezze dei sottoalberi al più 1.

7. Alberi bilanciati e AVL (definizione, caratteristiche, vantaggi/svantaggi).

AVL impone $|BF(v)| \leq 1$ con $BF(v) = h(sx) - h(dx)$ (fattore di bilanciamento); garantisce altezza $\Theta(\log n)$, ricerca/inserimento/cancellazione $\Theta(\log n)$; costo extra: rotazioni e metadati altezza/rank.

8. Differenze fra alberi binari bilanciati e perfettamente bilanciati.

Perfettamente bilanciato è più restrittivo (bilancio sul numero nodi), AVL più flessibile (bilancio sulle altezze) e mantenibile in modo efficiente.

9. Albero perfettamente bilanciato: definizione e costo delle operazioni.

Definizione sopra; altezza logaritmica quindi ricerca $\Theta(\log n)$, ma aggiornamenti possono richiedere ristrutturazioni globali costose.

10. Inserimento e ribilanciamento in un albero AVL.

Inserimento come in un albero binario di ricerca, poi risalita aggiornando il fattore di bilanciamento; se $|BF| = 2$ applica una rotazione (semplice o doppia).

11. **Alberi 2-3 (definizione, caratteristiche, altezza logaritmica).**
Ogni nodo interno ha 2 o 3 figli, tutte le foglie allo stesso livello; se n foglie: $2^h \leq n \leq 3^h$, quindi $h = \Theta(\log n)$.
12. **Definizione di albero binario di ricerca.**
Stessa definizione dell'albero binario di ricerca: ordine locale sinistra/destra che induce ordine globale con visita simmetrica.
13. **Come stampare i nodi di un albero binario di ricerca in ordine decrescente.**
Visita in-order inversa: destra, radice, sinistra.
14. **Alberi AVL: definizione, rotazioni, e bilanciamento.**
AVL: albero binario di ricerca con $|BF| \leq 1$; rotazioni semplici e doppie ripristinano il vincolo in tempo $\Theta(1)$ locale.
15. **Alberi 2-3: definizione, ricerca, inserimento, e numero di foglie.**
Ricerca segue chiavi guida interne, inserimento può causare split e propagazione verso l'alto; tutte le foglie alla stessa profondità.
16. **Alberi perfettamente bilanciati: definizione e proprietà.**
Differenza al più 1 nel numero di nodi dei due sottoalberi di ogni nodo; altezza $\Theta(\log n)$.
17. **Differenze tra alberi AVL, binari di ricerca e bilanciati.**
Un albero binario di ricerca non bilanciato può degradare ad altezza lineare; AVL è un albero binario di ricerca con bilanciamento in altezza; "bilanciato" è categoria generale (AVL, Red-Black, 2-3, ...).
18. **Cosa succede se un albero si trasforma in una lista (bilanciamento lineare).**
Altezza diventa $\Theta(n)$, quindi ricerca/inserimento/cancellazione diventano $\Theta(n)$.
19. **Problemi relativi agli alberi, come l'albero ricoprente minimo.**
Il problema dell'albero ricoprente minimo (MST) è un problema su grafi che produce un albero su tutti i vertici minimizzando il costo totale.
20. **Definizione di albero in termini di grafi (connesso, privo di cicli).**
Un albero è un grafo non orientato connesso e aciclico (equivalentemente con $n - 1$ archi).
21. **Struttura heap: definizione, rappresentazione come array posizionale.**
Max-heap: albero binario quasi completo con chiave padre $\geq$ chiavi figli; in array: figli di i in $2i + 1, 2i + 2$.
22. **Heapsort: implementazione, complessità, e utilizzo.**
Costruisci heap (bottom-up $\Theta(n)$), poi estrai massimo con swap+fix $n - 1$ volte: totale $\Theta(n \log n)$, in-place, non stabile.

6 Algoritmi di ordinamento

1. **Quando un algoritmo di ordinamento è detto stabile?**
Se elementi con chiave uguale mantengono l'ordine relativo iniziale.
2. **Il mergesort: funzionamento e complessità.**
Divide et impera: divide a metà, ordina ricorsivamente, fonde con merge lineare; tempo $\Theta(n \log n)$, spazio aggiuntivo tipico $\Theta(n)$.
3. **Differenze e somiglianze fra mergesort e quicksort.**
Entrambi divide et impera; MergeSort ha costo garantito $\Theta(n \log n)$ ma usa memoria extra, QuickSort è in-place ma ha worst-case $\Theta(n^2)$.

4. **Il quicksort: costo, casi migliori, peggiori e medi.**
Migliore/medio $\Theta(n \log n)$ con partizioni bilanciate o casuali; peggiore $\Theta(n^2)$ con partizioni estreme.
5. **Caso base di ricorsione nel quicksort.**
Sottoarray di lunghezza ≤ 1 (nel corso: condizione $\ell \geq r$ oppure $f - i \leq 1$).
6. **BubbleSort: definizione, funzionamento, costo temporale (caso migliore e peggiore).**
Scambia coppie adiacenti fuori ordine in più passate; best-case $\Theta(n)$ con controllo **swapped**, worst-case $\Theta(n^2)$.
7. **SelectionSort: definizione e funzionamento.**
A ogni passo seleziona il minimo della porzione non ordinata e lo scambia in posizione corrente; tempo $\Theta(n^2)$.
8. **InsertionSort: definizione e funzionamento.**
Costruisce prefisso ordinato inserendo ogni nuovo elemento nel punto corretto; best-case $\Theta(n)$, worst-case $\Theta(n^2)$.
9. **Heapsort: risorse necessarie, funzionamento e costo.**
In-place (spazio extra $\Theta(1)$ oltre array), build-heap + estrazioni: $\Theta(n \log n)$ in tutti i casi asintotici.
10. **Descrizione e funzionamento di BubbleSort, MergeSort, QuickSort, HeapSort.**
Bubble: scambi locali; Merge: divide+merge; Quick: partiziona su pivot; Heap: ordina tramite max-heap.
11. **Complessità temporale di MergeSort, QuickSort, e HeapSort.**
MergeSort $\Theta(n \log n)$, QuickSort medio $\Theta(n \log n)$ e pessimo $\Theta(n^2)$, HeapSort $\Theta(n \log n)$.
12. **Algoritmi di ordinamento che hanno prestazioni buone nel caso migliore.**
InsertionSort e BubbleSort ottimizzato hanno best-case lineare; Merge/Heap/Quick restano comunque almeno $\Theta(n \log n)$ nel caso tipico.
13. **Differenza tra MergeSort e QuickSort.**
MergeSort combina costi prevedibili e stabilità; QuickSort privilegia località/in-place ma dipende dalla qualità del pivot.
14. **Algoritmi di ordinamento non basati sui confronti: esempio RadixSort.**
RadixSort ordina per cifre usando sottoprocedura stabile (bucket/counting per cifra): evita lower bound dei confronti.
15. **Confronto sul numero di operazioni nei diversi algoritmi.**
Elementari: $\Theta(n^2)$ (Bubble/Selection/Insertion worst-case). Avanzati: $\Theta(n \log n)$ (Merge/Heap/Quick medio). Non a confronti: anche lineari in parametri favorevoli.
16. **Algoritmi che usano Divide et Impera (MergeSort, QuickSort).**
Schema: dividi istanza, risolvi sottoistanze ricorsive, combina risultati.

7 Strutture dati

1. **Struttura Union-Find: QuickFind, QuickUnion, compressione di cammino.**
QuickFind: FIND rapido, UNION costosa. QuickUnion: contrario. Con unione per rango e compressione di cammino ottieni costo ammortizzato molto basso (nelle note: fino a $\log^* n$, in letteratura classica $\alpha(n)$).

2. **Code di priorità: definizione, utilizzo e implementazione.**
Struttura dati astratta che mantiene elementi con priorità, operazioni principali INSERT, FINDMIN/MAX, EXTRACTMIN/MAX, DECREASEKEY; implementazione tipica con heap binario.
3. **Gestione delle collisioni nelle hash table: liste di collisione e indirizzamento aperto.**
Concatenamento esterno: bucket con liste. Indirizzamento aperto: scansione lineare/quadratica/hash doppio in tabella unica.
4. **Differenza tra coda FIFO e coda di priorità.**
FIFO estrae in ordine temporale di arrivo; coda con priorità estrae per priorità minima/massima.
5. **Relazione tra numero di nodi e foglie in un albero binario.**
Per un albero binario qualsiasi con n nodi e altezza h : $h + 1 \leq n \leq 2^{h+1} - 1$. In un binario pieno vale anche $f = i + 1$ (foglie vs nodi interni).
6. **Applicazioni delle code di priorità (es. Dijkstra).**
Dijkstra, Prim, scheduling, simulazioni a eventi discreti, gestione job con urgenza.
7. **Le code di priorità: definizione, operazioni, implementazione con heap.**
Con min-heap: INSERT/DECREASEKEY $\mathcal{O}(\log n)$, EXTRACTMIN $\mathcal{O}(\log n)$, FINDMIN $\mathcal{O}(1)$.
8. **Cos'è un mini heap e condizioni in cui è un heap?**
Min-heap: per ogni nodo, chiave padre $\leq$ chiavi figli + struttura di albero quasi completo.
9. **Relazione fra heap e array (vettore posizionale e rimozione della radice).**
Heap in array contiguo; rimozione radice: scambia con ultimo elemento, riduci dimensione logica, poi FIXHEAP dall'indice 0.
10. **Operazioni sugli heap (createHeap, risistema).**
CREATEHEAP bottom-up da $\lfloor n/2 \rfloor$ a 0; RISISTEMA/FIXHEAP ripristina proprietà facendo scendere la chiave lungo il cammino corretto.
11. **Compressione di cammino in Union-Find: quali nodi migliora?**
Migliora tutti i nodi sul cammino dal nodo cercato alla radice: dopo FIND puntano più direttamente alla radice.
12. **Strutture Union-Find: cosa sono, a cosa servono, quali operazioni ci puoi fare.**
Rappresentano partizioni in insiemi disgiunti; usi: connettività dinamica, Kruskal; operazioni: MAKESET, FIND, UNION.
13. **Tabella hash: definizione, gestione delle collisioni (esterna e interna), rehashing.**
Array indicizzato da $h(k)$. Collisioni: chaining o open addressing. Rehashing: quando $\alpha = n/m$ supera soglia (tipicamente $\approx 1/2$), si rialloca tabella più grande e reinserisce.

8 Analisi degli algoritmi

1. **Complessità temporale e spaziale degli algoritmi ($\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, etc.).**
Misurano crescita asintotica di tempo e memoria rispetto alla dimensione input, ignorando costanti e termini minori.
2. **Differenza tra confronti e tempo negli ordinamenti.**
I confronti sono una metrica astratta; il tempo reale dipende anche da movimenti dati, cache, allocazioni, costo del confronto.

3. **Algoritmi greedy: definizione e applicazioni a problemi come lo zaino.**
 Scelte locali irrevocabili. Funzionano per il problema dell'albero ricoprente minimo (MST) e per Dijkstra (con ipotesi), ma per zaino 0/1 il greedy non garantisce ottimo (ottimo invece per zaino frazionario).
4. **Costo ammortizzato in strutture dati.**
 Costo medio per operazione su una sequenza, includendo operazioni occasionalmente costose (es. rehash), ma mantenendo costo medio basso.
5. **Lower bound degli algoritmi di ordinamento basati su confronti.**
 Ogni ordinamento a confronti richiede nel caso peggiore almeno $\Omega(n \log n)$ confronti.
6. **Uso di tecniche come Divide et Impera e programmazione dinamica.**
 Divide et impera: sottoproblemi indipendenti (MergeSort/QuickSort). Programmazione dinamica: sottoproblemi sovrapposti + memoizzazione/tabulazione.
7. **Cos'è l'analisi del costo medio?**
 Media del costo rispetto a una distribuzione degli input (caso medio), diversa dal caso peggiore e dall'analisi ammortizzata.
8. **Cosa si intende per costo ammortizzato?**
 Costo totale di una sequenza diviso numero operazioni; garantisce limite per operazione "in media sulla sequenza".
9. **Algoritmi greedy: esempi ottimali e non.**
 Ottimali: Kruskal, Prim, Dijkstra (pesi non negativi). Non ottimali in generale: zaino 0/1 con criterio locale semplice.
10. **Differenze fra programmazione dinamica, divide et impera e algoritmi greedy.**
 Greedy: scelta locale senza revisione; Divide et Impera: divide e combina sottoproblemi indipendenti; programmazione dinamica: risolve una volta sottoproblemi sovrapposti e riusa risultati.
11. **Equazione di ricorrenza.**
 È la relazione che esprime il costo ricorsivo, es. $T(n) = aT(n/b) + f(n)$; si risolve con sostituzione/albero di ricorsione/Master theorem.

9 Mini-schede di complessità utili per ripasso

- MergeSort : $\Theta(n \log n)$ tempo, $\Theta(n)$ spazio ausiliario tipico.
- QuickSort : $\Theta(n \log n)$ medio, $\Theta(n^2)$ peggiore, stack medio $\Theta(\log n)$.
- HeapSort : $\Theta(n \log n)$ tempo, in-place.
- Dijkstra : $\mathcal{O}((n + m) \log n)$ con heap binario.
- Bellman-Ford : $\Theta(nm)$.
- Floyd-Warshall : $\Theta(n^3)$ tempo, $\Theta(n^2)$ spazio.
- Kruskal : $\mathcal{O}(m \log m)$ tipico.
- Prim : $\mathcal{O}(m \log n)$ con heap e liste di adiacenza.